CSCI 441/541 · APPLIED DEEP LEARNING · ASSIGNMENT 1 + PROJECT 1 REPORT

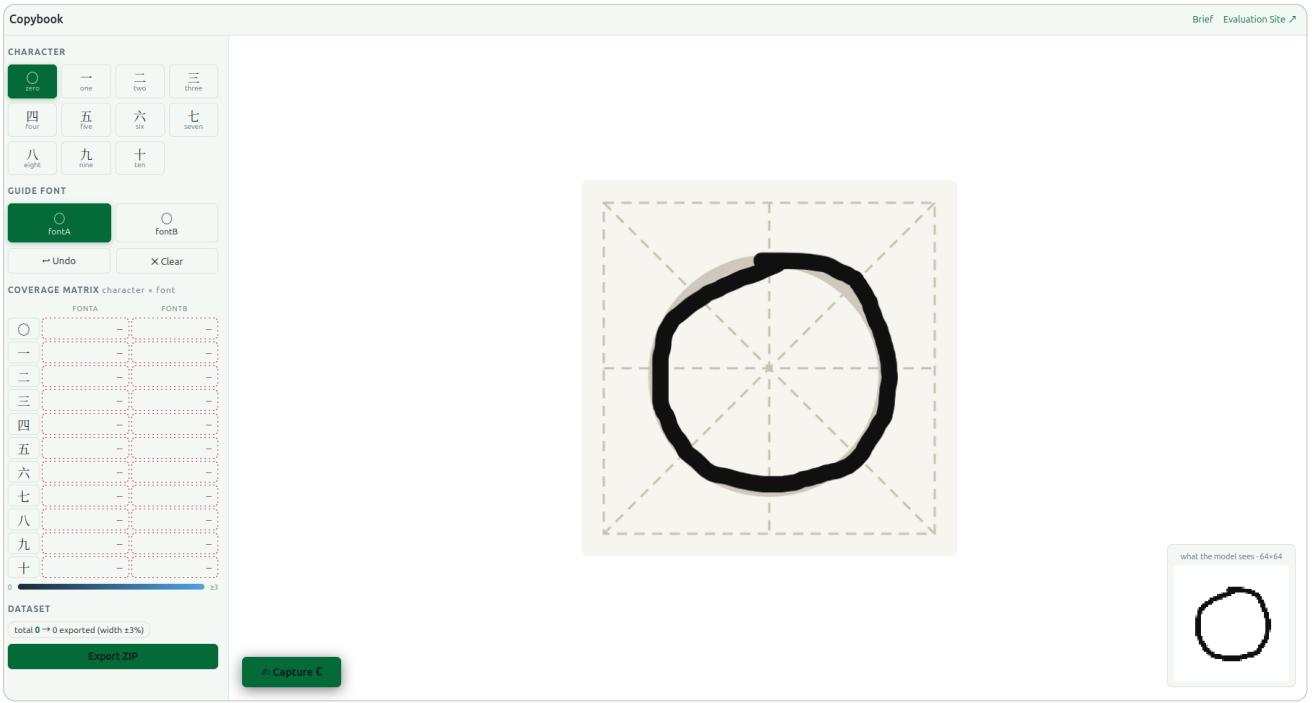
cg-linear and cg-mlp

Student: Caden Graf caden.graf@ndus.edu · **Generated:** 2026-10-19

Overview

I built a classifier for 11 characters, using a linear softmax model and then training a two-hidden-layer MLP. The linear model used a 16x16 pooled grayscale images, while the MLP used 8x8 pooled images with dark ink represented by larger values. On the hidden set, accuracy rose from 81.0% to 90.5%. The most important things that I learned is that the MLP could recognize these characters more accurately despite using a lower-resolution image representation. AI was used to help confirm answers, cleanup report, and write augment dataset script.

Dataset & collection



How many raw captures per character did you collect, and how did you keep the classes balanced? Any capture you redid and why?

This is just an example of using the copybook for collecting data. I traced each of the fonts for every character 7 times each so that they were balanced. I originally traced 2 for each and trained a linear classifier. The results were not good enough so I went back and traced 5 more for each of the pictures. Basically, I just didn't have enough data to get good results.

Assignment — linear softmax baseline

Linear evidence block 1

```
(.venv) megrad@caden-graf:~/Documents/GitHub/CSCI541/homeworks/hw1$ python partB.py
loaded 5082 traced images ({np.int64(0): np.int64(462), np.int64(1): np.int64(462), np.int64(2): np.int64(462), np.int64(3): np.int64(462), np.int
64(4): np.int64(462), np.int64(5): np.int64(462), np.int64(6): np.int64(462), np.int64(7): np.int64(462), np.int64(8): np.int64(462), np.int64(9):
 np.int64(462), np.int64(10): np.int64(462)})
[A] epoch   0  train acc 0.954
[A] epoch  20  train acc 0.996
[A] epoch  40  train acc 0.994
[A] epoch  59  train acc 0.999
[A] wrote linear_model.onnx  (2827 params) — submit for the Assignment
```

Description

Shows the dataset has 5082 images with 462 per class. Training accuracy rose from 0.954 at epoch 0 to 0.999 at epoch 59. This accuracy doesn't necessarily mean it can handle unfamiliar images. It is just saying the accuracy on this specific dataset. The hidden accuracy is a better descriptor for unfamiliar images.

Linear evidence block 3

Report #124 — cg-linear / my-linear-v1 10/13/2026, 2:10:11 PM
public set (n=400) **80.0%**
hidden set (n=2200) **81.0%**

params **2,829** · opset 17 · NCHW · latency median **0.01 ms**, p95 0.01 ms · score **0.702**

HIDDEN-SET SLICES

fontA (n=550) **68.2%**
bold (n=496) **83.7%**
fontB (n=550) **93.8%**
fontC (n=550) **87.3%**
fontD (n=550) **74.6%**
rotated (n=686) **82.9%**

CONFUSION MATRIX

truth↓ pred→	〇	一	二	三	四	五	六	七	八	九	十
〇	200	0	0	0	0	0	0	0	0	0	0
一	0	173	0	0	0	0	10	16	0	0	1
二	0	0	151	0	8	13	0	0	26	2	0
三	0	0	31	125	3	25	0	4	0	0	12
四	0	0	2	0	170	4	22	0	0	2	0
五	0	0	2	0	0	134	0	0	0	48	16
六	0	0	0	0	1	13	167	9	5	3	2
七	0	0	0	0	0	1	0	124	0	2	73
八	0	0	0	0	9	8	3	0	150	29	1
九	0	0	0	0	5	0	0	6	1	188	0
十	0	0	0	0	0	0	0	1	0	0	199

most confused: 七→十 ×73, 五→九 ×48, 三→二 ×31

Description

The report ID.

Project — the MLP

Experiment block 1

```
[B] representation prototype: 64 dims (mirror the SAME preprocessing inside MLP.forward, TODO-B3)
[B] architecture accepted: 42379 parameters (85% of budget)
[B] epoch   0  val acc 0.956
[B] epoch  20  val acc 0.997
[B] epoch  40  val acc 1.000
[B] epoch  60  val acc 1.000
[B] epoch  80  val acc 1.000
[B] epoch 100  val acc 1.000
[B] epoch 120  val acc 1.000
[B] epoch 140  val acc 1.000
[B] epoch 160  val acc 1.000
[B] epoch 180  val acc 1.000
[B] epoch 199  val acc 1.000
/home/megrad/Documents/GitHub/CSCI541/homeworks/hw1/partB.py:84: DeprecationWarning: You are using the legacy TorchScript-base
d ONNX export. Starting in PyTorch 2.9, the new torch.export-based ONNX exporter has become the default. Learn more about the
new export logic: https://docs.pytorch.org/docs/stable/onnx_export.html. For exporting control flow: https://pytorch.org/tutor
ials/beginner/onnx/export_control_flow_model_to_onnx_tutorial.html
  torch.onnx.export(model, torch.zeros(1, 1, 64, 64), path,
[B] mlp_model.onnx verified — submit at /board.html and keep the submission ID for your report
(.venv) megrad@caden-graf:~/Documents/GitHub/CSCI541/homeworks/hw1$ python partB.py
loaded 5082 traced images ({np.int64(0): np.int64(462), np.int64(1): np.int64(462), np.int64(2): np.int64(462), np.int64(3): n
p.int64(462), np.int64(4): np.int64(462), np.int64(5): np.int64(462), np.int64(6): np.int64(462), np.int64(7): np.int64(462),
np.int64(8): np.int64(462), np.int64(9): np.int64(462), np.int64(10): np.int64(462)})
[A] epoch   0  train acc 0.954
[A] epoch  20  train acc 0.996
[A] epoch  40  train acc 0.994
[A] epoch  59  train acc 0.999
[A] wrote linear_model.onnx  (2827 params) — submit for the Assignment
[B] representation prototype: 64 dims (mirror the SAME preprocessing inside MLP.forward, TODO-B3)
[B] architecture accepted: 30091 parameters (60% of budget)
[B] epoch   0  val acc 0.706
[B] epoch  20  val acc 0.921
[B] epoch  40  val acc 0.954
[B] epoch  60  val acc 0.950
[B] epoch  80  val acc 0.981
[B] epoch 100  val acc 0.962
[B] epoch 120  val acc 0.970
[B] epoch 140  val acc 0.976
[B] epoch 160  val acc 0.983
[B] epoch 180  val acc 0.983
[B] epoch 199  val acc 0.972
/home/megrad/Documents/GitHub/CSCI541/homeworks/hw1/partB.py:84: DeprecationWarning: You are using the legacy TorchScript-base
d ONNX export. Starting in PyTorch 2.9, the new torch.export-based ONNX exporter has become the default. Learn more about the
new export logic: https://docs.pytorch.org/docs/stable/onnx_export.html. For exporting control flow: https://pytorch.org/tutor
ials/beginner/onnx/export_control_flow_model_to_onnx_tutorial.html
  torch.onnx.export(model, torch.zeros(1, 1, 64, 64), path,
[B] mlp_model.onnx verified — submit at /board.html and keep the submission ID for your report
```

Description

Change the pool kernel from 8 to 16, and fc1 input size from 8 * 8 to 4 * 4. Testing if less image detail hurt validation accuracy. The training is slower. The val acc converges at around 0.983 it doesn't reach 1.00. Less image detail hurts validation accuracy.

Experiment block 2

```
[B] representation prototype: 64 dims (mirror the SAME preprocessing inside MLP.forward, TODO-B3)
[B] architecture accepted: 42379 parameters (85% of budget)
[B] epoch   0  val acc 0.955
[B] epoch  20  val acc 0.998
[B] epoch  40  val acc 1.000
[B] epoch  60  val acc 1.000
[B] epoch  80  val acc 1.000
[B] epoch 100  val acc 0.999
[B] epoch 120  val acc 1.000
[B] epoch 140  val acc 1.000
[B] epoch 160  val acc 1.000
[B] epoch 180  val acc 1.000
[B] epoch 199  val acc 1.000
/home/megrad/Documents/GitHub/CSCI541/homeworks/hw1/partB.py:84: DeprecationWarning: You are using the legacy TorchScript-base
d ONNX export. Starting in PyTorch 2.9, the new torch.export-based ONNX exporter has become the default. Learn more about the
new export logic: https://docs.pytorch.org/docs/stable/onnx_export.html. For exporting control flow: https://pytorch.org/tutor
ials/beginner/onnx/export_control_flow_model_to_onnx_tutorial.html
  torch.onnx.export(model, torch.zeros(1, 1, 64, 64), path,
[B] mlp_model.onnx verified — submit at /board.html and keep the submission ID for your report
(.venv) megrad@caden-graf:~/Documents/GitHub/CSCI541/homeworks/hw1$ python partB.py
loaded 5082 traced images ({np.int64(0): np.int64(462), np.int64(1): np.int64(462), np.int64(2): np.int64(462), np.int64(3): n
p.int64(462), np.int64(4): np.int64(462), np.int64(5): np.int64(462), np.int64(6): np.int64(462), np.int64(7): np.int64(462),
np.int64(8): np.int64(462), np.int64(9): np.int64(462), np.int64(10): np.int64(462)})
[A] epoch   0  train acc 0.954
[A] epoch  20  train acc 0.996
[A] epoch  40  train acc 0.994
[A] epoch  59  train acc 0.999
[A] wrote linear_model.onnx  (2827 params) — submit for the Assignment
[B] representation prototype: 64 dims (mirror the SAME preprocessing inside MLP.forward, TODO-B3)
[B] architecture accepted: 21771 parameters (44% of budget)
[B] epoch   0  val acc 0.952
[B] epoch  20  val acc 0.996
[B] epoch  40  val acc 1.000
[B] epoch  60  val acc 1.000
[B] epoch  80  val acc 1.000
[B] epoch 100  val acc 0.999
[B] epoch 120  val acc 1.000
[B] epoch 140  val acc 1.000
[B] epoch 160  val acc 1.000
[B] epoch 180  val acc 1.000
[B] epoch 199  val acc 1.000
/home/megrad/Documents/GitHub/CSCI541/homeworks/hw1/partB.py:84: DeprecationWarning: You are using the legacy TorchScript-base
d ONNX export. Starting in PyTorch 2.9, the new torch.export-based ONNX exporter has become the default. Learn more about the
new export logic: https://docs.pytorch.org/docs/stable/onnx_export.html. For exporting control flow: https://pytorch.org/tutor
ials/beginner/onnx/export_control_flow_model_to_onnx_tutorial.html
  torch.onnx.export(model, torch.zeros(1, 1, 64, 64), path,
[B] mlp_model.onnx verified — submit at /board.html and keep the submission ID for your report
```

Description

Change the fc1 from 256 to 128 and fc2 input from 256 to 128 as well. Is the smaller model as accurate? It shows that the val accuracy is basically the same.

Experiment block 3

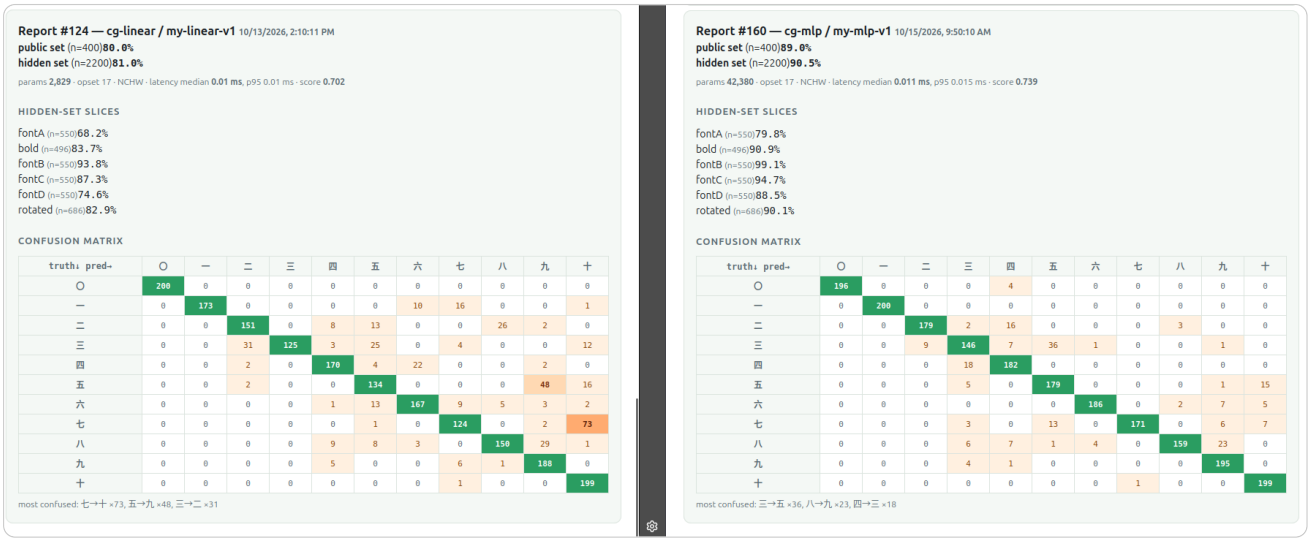
```
[B] representation prototype: 64 dims (mirror the SAME preprocessing inside MLP.forward, TODO-B3)
[B] architecture accepted: 42379 parameters (85% of budget)
[B] epoch   0  val acc 0.959
[B] epoch  20  val acc 0.998
[B] epoch  40  val acc 1.000
[B] epoch  60  val acc 1.000
[B] epoch  80  val acc 1.000
[B] epoch 100  val acc 1.000
[B] epoch 120  val acc 1.000
[B] epoch 140  val acc 1.000
[B] epoch 160  val acc 1.000
[B] epoch 180  val acc 1.000
[B] epoch 199  val acc 1.000
/home/megrad/Documents/GitHub/CSCI541/homeworks/hw1/partB.py:84: DeprecationWarning: You are using the legacy TorchScript-base
d ONNX export. Starting in PyTorch 2.9, the new torch.export-based ONNX exporter has become the default. Learn more about the
new export logic: https://docs.pytorch.org/docs/stable/onnx_export.html. For exporting control flow: https://pytorch.org/tutor
ials/beginner/onnx/export_control_flow_model_to_onnx_tutorial.html
  torch.onnx.export(model, torch.zeros(1, 1, 64, 64), path,
[B] mlp_model.onnx verified — submit at /board.html and keep the submission ID for your report
(.venv) megrad@caden-graf:~/Documents/GitHub/CSCI541/homeworks/hw1$ python partB.py
loaded 5082 traced images ({np.int64(0): np.int64(462), np.int64(1): np.int64(462), np.int64(2): np.int64(462), np.int64(3): n
p.int64(462), np.int64(4): np.int64(462), np.int64(5): np.int64(462), np.int64(6): np.int64(462), np.int64(7): np.int64(462),
np.int64(8): np.int64(462), np.int64(9): np.int64(462), np.int64(10): np.int64(462)})
[A] epoch   0  train acc 0.954
[A] epoch  20  train acc 0.996
[A] epoch  40  train acc 0.994
[A] epoch  59  train acc 0.999
[A] wrote linear_model.onnx  (2827 params) — submit for the Assignment
[B] representation prototype: 64 dims (mirror the SAME preprocessing inside MLP.forward, TODO-B3)
[B] architecture accepted: 19467 parameters (39% of budget)
[B] epoch   0  val acc 0.899
[B] epoch  20  val acc 0.987
[B] epoch  40  val acc 0.997
[B] epoch  60  val acc 0.998
[B] epoch  80  val acc 0.998
[B] epoch 100  val acc 0.998
[B] epoch 120  val acc 0.998
[B] epoch 140  val acc 0.998
[B] epoch 160  val acc 0.999
[B] epoch 180  val acc 0.998
[B] epoch 199  val acc 1.000
/home/megrad/Documents/GitHub/CSCI541/homeworks/hw1/partB.py:82: DeprecationWarning: You are using the legacy TorchScript-base
d ONNX export. Starting in PyTorch 2.9, the new torch.export-based ONNX exporter has become the default. Learn more about the
new export logic: https://docs.pytorch.org/docs/stable/onnx_export.html. For exporting control flow: https://pytorch.org/tutor
ials/beginner/onnx/export_control_flow_model_to_onnx_tutorial.html
  torch.onnx.export(model, torch.zeros(1, 1, 64, 64), path,
[B] mlp_model.onnx verified — submit at /board.html and keep the submission ID for your report
```

Description

When I changed the depth of the MLP by removing the fc2 layer and connect the 256-unit fc1 directly to a 11-output layer. Trying to see if the second hidden layer helps. It seems like when I removed that layer it took longer to converge to the 1.000 val acc.

Evaluation Site results

Submission IDs



Report #124 — cg-linear / my-linear-v1 10/13/2026, 2:10:11 PM
public set (n=400) **80.0%**
hidden set (n=2200) **81.0%**
params 2,829 · opset 17 · NCHW · latency median **0.01 ms**, p95 0.01 ms · score **0.702**

HIDDEN-SET SLICES

fontA (n=550)	68.2%
bold (n=496)	83.7%
fontB (n=550)	93.8%
fontC (n=550)	87.3%
fontD (n=550)	74.6%
rotated (n=686)	82.9%

CONFUSION MATRIX

truth↓ pred→	○	一	二	三	四	五	六	七	八	九	十
○	200	0	0	0	0	0	0	0	0	0	0
一	0	173	0	0	0	0	10	16	0	0	1
二	0	0	151	0	8	13	0	0	26	2	0
三	0	0	31	125	3	25	0	4	0	0	12
四	0	0	2	0	170	4	22	0	0	2	0
五	0	0	2	0	0	134	0	0	0	48	16
六	0	0	0	0	1	13	167	9	5	3	2
七	0	0	0	0	0	1	0	124	0	2	73
八	0	0	0	0	9	8	3	0	150	29	1
九	0	0	0	0	5	0	0	6	1	188	0
十	0	0	0	0	0	0	0	1	0	0	199

most confused: 七→十 ×73, 五→九 ×48, 三→二 ×31

Report #160 — cg-mlp / my-mlp-v1 10/15/2026, 9:50:10 AM
public set (n=400) **89.0%**
hidden set (n=2200) **90.5%**
params 42,380 · opset 17 · NCHW · latency median **0.011 ms**, p95 0.015 ms · score **0.739**

HIDDEN-SET SLICES

fontA (n=550)	79.8%
bold (n=496)	90.9%
fontB (n=550)	99.1%
fontC (n=550)	94.7%
fontD (n=550)	88.5%
rotated (n=686)	90.1%

CONFUSION MATRIX

truth↓ pred→	○	一	二	三	四	五	六	七	八	九	十
○	196	0	0	0	4	0	0	0	0	0	0
一	0	200	0	0	0	0	0	0	0	0	0
二	0	0	179	2	16	0	0	0	3	0	0
三	0	0	9	146	7	36	1	0	0	1	0
四	0	0	0	18	182	0	0	0	0	0	0
五	0	0	0	5	0	179	0	0	0	1	15
六	0	0	0	0	0	0	186	0	2	7	5
七	0	0	0	3	0	13	0	171	0	6	7
八	0	0	0	6	7	1	4	0	159	23	0
九	0	0	0	4	1	0	0	0	0	195	0
十	0	0	0	0	0	0	0	1	0	0	199

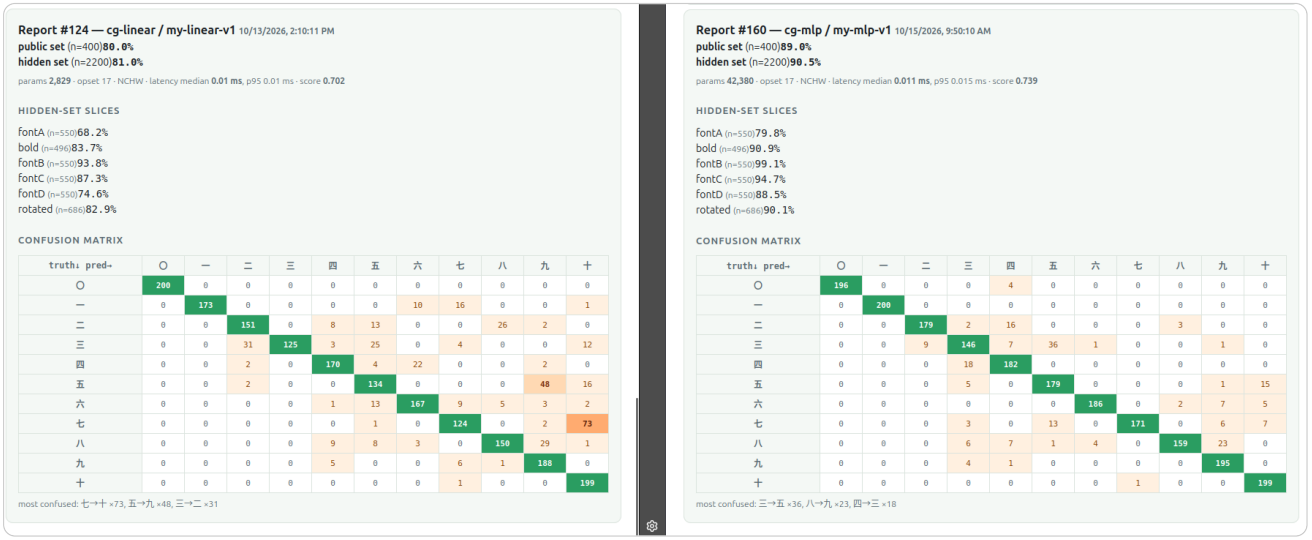
most confused: 三→五 ×36, 八→九 ×23, 四→三 ×18

List both submission IDs (linear and MLP) and their scores.

linear: id:124, hidden_acc: 81.0, score: 0.702

MLP: id: 160, hidden_acc: 90.5%, score: 0.739

Hidden-dataset results & analysis



Report #124 — cg-linear / my-linear-v1 10/13/2026, 2:10:11 PM
public set (n=400) 80.0%
hidden set (n=2200) 81.0%
params 2,829 · opset 17 · NCHW · latency median 0.01 ms, p95 0.01 ms · score 0.702

HIDDEN-SET SLICES

fontA (n=550)	68.2%
bold (n=496)	83.7%
fontB (n=550)	93.8%
fontC (n=550)	87.3%
fontD (n=550)	74.6%
rotated (n=686)	82.9%

CONFUSION MATRIX

truth↓ pred→	○	一	二	三	四	五	六	七	八	九	十
○	200	0	0	0	0	0	0	0	0	0	0
一	0	173	0	0	0	0	10	16	0	0	1
二	0	0	151	0	8	13	0	0	26	2	0
三	0	0	31	125	3	25	0	4	0	0	12
四	0	0	2	0	170	4	22	0	0	2	0
五	0	0	2	0	0	134	0	0	0	48	16
六	0	0	0	0	1	13	167	9	5	3	2
七	0	0	0	0	0	1	0	124	0	2	73
八	0	0	0	0	9	8	3	0	150	29	1
九	0	0	0	0	5	0	0	6	1	188	0
十	0	0	0	0	0	0	0	1	0	0	199

most confused: 七→十 ×73, 五→九 ×48, 三→二 ×31

Report #160 — cg-mlp / my-mlp-v1 10/15/2026, 9:50:10 AM
public set (n=400) 89.0%
hidden set (n=2200) 90.5%
params 42,380 · opset 17 · NCHW · latency median 0.011 ms, p95 0.015 ms · score 0.739

HIDDEN-SET SLICES

fontA (n=550)	79.8%
bold (n=496)	90.9%
fontB (n=550)	99.1%
fontC (n=550)	94.7%
fontD (n=550)	88.5%
rotated (n=686)	90.1%

CONFUSION MATRIX

truth↓ pred→	○	一	二	三	四	五	六	七	八	九	十
○	196	0	0	0	4	0	0	0	0	0	0
一	0	200	0	0	0	0	0	0	0	0	0
二	0	0	179	2	16	0	0	0	3	0	0
三	0	0	9	146	7	36	1	0	0	1	0
四	0	0	0	18	182	0	0	0	0	0	0
五	0	0	0	5	0	179	0	0	0	1	15
六	0	0	0	0	0	0	186	0	2	7	5
七	0	0	0	3	0	13	0	171	0	6	7
八	0	0	0	6	7	1	4	0	159	23	0
九	0	0	0	4	1	0	0	0	0	195	0
十	0	0	0	0	0	0	0	1	0	0	199

most confused: 三→五 ×36, 八→九 ×23, 四→三 ×18

Analysis (graded): which font slice or confusion pair is weakest, and why do you think that happens? What would you try next?

The weakest slice is fontA at 68.2% for my linear model and 79.8% for MLP. I think that happened because either my drawings were too different from each other so they couldn't learn valuable features. Or I just don't have enough data for them to learn fontA better or it overfitted to other fonts. FontA may have stroke shapes or spacing that is different from my drawings. I would add more examples. The weakest confusion pair is when it is supposed to be a three it mistakes it for five in the MLP. It happens because five has three strokes going horizontal across similar to three. Something that would help would be more data would be beneficial as well augmentations.

Comparison & conclusion

Linear vs MLP on the hidden set (accuracy, parameters, latency)?

Linear reached 81.0% on the hidden set with 2,829 params, 0.01ms latency, and 0.702 score. The MLP reached 90.5% has 42,380 params, 0.011 ms latency, and 0.739 score. I believe that the extra params on the MLP was worth it to get the better results. The MLP has better results on almost every single character but does slightly worse on O and same on +. So I believe that it is worth it.

Most helpful change, and what did not help?

The single best change that I did was adding augmentations to the dataset. Also, adding more images to the dataset helped the hidden accuracy a lot. Something that I tried right away that wasn't good was very little images and no augmentations. My onnx models were not passing before I added more imagery and augmentations.

Appendix A — sample_mlp.py

```
"""Self-contained script for the linear baseline and final two-hidden-layer MLP."""

import io
import json
import zipfile
import numpy as np
from PIL import Image
import importlib.util, subprocess, sys
if importlib.util.find_spec("onnx") is None:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "onnx"])
import onnx
from onnx import helper, TensorProto

GLYPHS = "〇一二三四五六七八九" + "十"      # class order = label ids 0..10
N_CLASSES = 11
INPUT_SHAPE = (1, 1, 64, 64)               # model contract: NCHW float32 [0,1]
def load_copybook(dataset_path):
    """Accepts a Copybook export zip or an unpacked directory.
    Returns (images float32 [n,64,64] in [0,1], labels int64 [n])."""
    if dataset_path.endswith(".zip"):
        zf = zipfile.ZipFile(dataset_path)
        meta = json.loads(zf.read("labels.json") if "labels.json" in zf.namelist()
                          else zf.read("manifest.json"))
        if isinstance(meta, dict):                          # copybook layout
            meta = meta.get("labels") or meta.get("shots")
        shots = []
        for m in meta:                                      # manfifest order
            name = f"shots/{m['file']}"
            img = Image.open(io.BytesIO(zf.read(name))).convert("L")
            shots.append(np.asarray(img, np.float32) / 255.0)
        return (np.stack(shots),
                np.array([m["label"] for m in meta], dtype=np.int64))
    raise SystemExit("point --dataset at your Copybook export zip")

def softmax(z):
    """Stable softmax (lecture 7 recipe). Provided."""
    z = z - z.max(axis=1, keepdims=True)
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)

D_A = 16 * 16          # representation width: 16x16 average-pooled grayscale
def part_a_features(images):
    """64x64 -> 16x16 average pool, ink positive, scaled to [-1,1]."""
    x = images.reshape(-1, 16, 4, 16, 4).mean(axis=(2, 4))
    return ((0.5 - x) * 2.0).reshape(len(images), -1)   # ink dark: invert
def part_a_train(X, y, epochs=60, lr=0.5):
    n, d = X.shape
    W = np.zeros((d, N_CLASSES), np.float32)    # zero init: sanity check below
    b = np.zeros(N_CLASSES, np.float32)

    n_check = min(64, n)   # works even with a tiny first dataset
    p0 = softmax(X[:n_check] @ W + b)
    loss0 = -np.log(p0[np.arange(n_check), y[:n_check]] + 1e-12).mean()
    assert abs(loss0 - np.log(N_CLASSES)) < 1e-3, "softmax/CE sanity failed"
    for ep in range(epochs):
        perm = np.random.default_rng(ep).permutation(n)
        for i in range(0, n, 64):
            j = perm[i:i + 64]
```

```
            Xb, yb = X[j], y[j]
            # ▼ blank A1 — affine forward · Lecture 7
            logits = Xb @ W + b
            p = softmax(logits)

            # ▼ blank A2 — softmax cross-entropy gradient · Lecture 7
            p[np.arange(len(yb)), yb] -= 1.0
            grad_W = Xb.T @ p / len(yb)
            grad_b = p.mean(axis=0)
            # ▼ blank A3 — SGD update · Lecture 8
            W -= lr * grad_W
            b -= lr * grad_b

        if ep % 20 == 0 or ep == epochs - 1:
            acc = (softmax(X @ W + b).argmax(1) == y).mean()
            print(f"[A] epoch {ep:3d}  train acc {acc:.3f}")
    return W, b
def export_linear(W, b, path="linear_model.onnx"):

    def tensor(name, arr):
        return helper.make_tensor(name, TensorProto.FLOAT, arr.shape,
                                  arr.astype(np.float32).tobytes(), raw=True)
    shape = helper.make_tensor("fshape", TensorProto.INT64, [2],
                               np.array([1, D_A], np.int64).tobytes(), raw=True)
    nodes = [
        helper.make_node("AveragePool", ["image"], ["p"], kernel_shape=[4, 4], strides=[4,
4]),
        helper.make_node("Constant", [], ["half"],
                         value=tensor("half", np.array(0.5, np.float32))),
        helper.make_node("Sub", ["half", "p"], ["inv"]),          # 0.5 - pool
        helper.make_node("Constant", [], ["two"],
                         value=tensor("two", np.array(2.0, np.float32))),
        helper.make_node("Mul", ["inv", "two"], ["x"]),           # scale
        helper.make_node("Reshape", ["x", "fshape"], ["f"]),      # [1, 256]
        helper.make_node("Gemm", ["f", "Wt", "bt"], ["logits"]),
    ]
    graph = helper.make_graph(
        nodes, "linear_softmax",
        [helper.make_tensor_value_info("image", TensorProto.FLOAT, INPUT_SHAPE)],
        [helper.make_tensor_value_info("logits", TensorProto.FLOAT,
                                       [1, N_CLASSES])],
        [tensor("Wt", W.astype(np.float32)),
         tensor("bt", b), shape])
    model = helper.make_model(graph, opset_imports=[
        helper.make_opsetid("", 17)])
    model.ir_version = 9
    onnx.checker.check_model(model)
    onnx.save(model, path)
    print(f"[A] wrote {path}  ({W.size + b.size} params) — submit for the Assignment")

# — PART B —
import torch
from torch import nn
import torch.nn.functional as F
import numpy as np
from PIL import Image
import json

BUDGET = 50_000
```

```
def part_b_features(images):
    """images [n,64,64] in [0,1]  ->  features [n, d], bounded."""
    # Average each 8x8 block to keep broader stroke shapes.
    # ▼ blank B1 — representation · New idea · this project

    pooled = images.reshape(-1, 8, 8, 8, 8).mean(axis=(2, 4))
    return (1.0 - pooled).reshape(len(images), -1)

def count_params(model):
    # ▼ blank B2 — parameter accounting · Lecture 9

    return sum(p.numel() for p in model.parameters())



class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        # ▼ blank B3a — declare your layers · Lecture 9
        self.fc1 = nn.Linear(8 * 8, 256)
        self.fc2 = nn.Linear(256, 96)
        self.fc3 = nn.Linear(96, 11)   # N_CLASSES = 11
    def forward(self, x):                    # x: [1,1,64,64]
        # ▼ blank B3b — forward: preprocessing in-graph + layers · Lecture 9

        x = F.avg_pool2d(x, kernel_size=8)
        x = (1.0 - x).reshape(x.shape[0], -1)
        x = torch.tanh(self.fc1(x))
        x = torch.tanh(self.fc2(x))
        x = self.fc3(x)
        return x

def init_weights(m):
    # ▼ blank B4 — initialization policy · Lecture 9

    if isinstance(m, nn.Linear):
        torch.nn.init.xavier_uniform_(m.weight, gain=nn.init.calculate_gain("tanh"))
        m.bias.data.fill_(0.01)

def part_b_train(model, images, y, imgs_val, yval, epochs=200, lr=0.1, batch=32):
    """Feed RAW images [n,1,64,64]: the model preprocesses inside forward,
    exactly like the exported graph will (Lecture 9 encapsulation)."""
    X = torch.from_numpy(images.reshape(-1, 1, 64, 64))
    XvT = torch.from_numpy(imgs_val.reshape(-1, 1, 64, 64))
    yt = torch.from_numpy(y)
    opt = torch.optim.SGD(model.parameters(), lr=lr)   # Lecture 9 optimizer
    loss_fn = nn.CrossEntropyLoss()
    rng = np.random.default_rng(0)
    for ep in range(epochs):
        model.train()
        perm = rng.permutation(len(X))
        for i in range(0, len(perm), batch):
            j = perm[i:i + batch]
            # ▼ blank B5 — the four-step training step · Lecture 9

            opt.zero_grad()
            logits = model(X[j])
            loss = loss_fn(logits, yt[j])
            loss.backward()
```

```
            opt.step()

        model.eval()
        with torch.no_grad():
            val_acc = (model(XvT).argmax(1)
                       == torch.from_numpy(yval)).float().mean().item()
        if ep % 20 == 0 or ep == epochs - 1:
            print(f"[B] epoch {ep:3d}  val acc {val_acc:.3f}")
def export_mlp(model, path="mlp_model.onnx"):
    model.eval()
    torch.onnx.export(model, torch.zeros(1, 1, 64, 64), path,
                      input_names=["image"], output_names=["logits"],
                      opset_version=17, dynamo=False)
# — run —
if __name__ == "__main__":
    import argparse
    ap = argparse.ArgumentParser()
    ap.add_argument("--dataset", default="/home/megrad/Documents/GitHub/CSCI541/
pj1_combined_10aug.zip", help="your Copybook export")
    ap.add_argument("--val", default=None, help="optional manifest dir, e.g. the public
set")
    args, _unknown = ap.parse_known_args()   # Colab injects -f kernel.json; ignore it
    images, labels = load_copybook(args.dataset)
    print(f"loaded {len(images)} traced images "
          f"({dict(zip(*np.unique(labels, return_counts=True)))})")
# — Part A —
    XA = part_a_features(images)
    W, b = part_a_train(XA, labels)
    export_linear(W, b)
# — Part B —
    XB = part_b_features(images)                        # TODO-B1 prototype
    assert XB.ndim == 2 and np.abs(XB).max() <= 1.0 + 1e-5, \
        "representation must be a bounded [n, d] matrix"
    print(f"[B] representation prototype: {XB.shape[1]} dims "
          f"(mirror the SAME preprocessing inside MLP.forward, TODO-B3)")
    if args.val:                                          # public set as validation
        vm = json.load(open(f"{args.val}/manifest.json"))
        vi = np.stack([np.asarray(Image.open(f"{args.val}/{m['file']}").convert('L'),
                                  np.float32) / 255.0 for m in vm])
        vl = np.array([m["label"] for m in vm], np.int64)
    else:
        cut = max(8, len(images) // 5)
        vi, vl = images[:cut], labels[:cut]
    model = MLP()                                         # TODO-B3
    model.apply(init_weights)                             # TODO-B4
    n_params = count_params(model)                        # TODO-B2
    assert n_params <= BUDGET, f"{n_params} params exceeds the {BUDGET} budget"
    print(f"[B] architecture accepted: {n_params} parameters "
          f"({100 * n_params / BUDGET:.0f}% of budget)")
    part_b_train(model, images, labels, vi, vl)          # TODO-B5
    export_mlp(model)                                     # provided
    import onnx
    m = onnx.load("mlp_model.onnx")
    onnx.checker.check_model(m)
    dims = [d.dim_value for d in m.graph.input[0].type.tensor_type.shape.dim]
    assert dims == [1, 1, 64, 64], f"input shape {dims} — export block was modified?"
    print("[B] mlp_model.onnx verified — submit at /board.html and keep the "
          "submission ID for your report")
```